Absolutely. I'm treating `project-spec.txt` as the **primary source of truth**, and the "Building AI Agents" material as supporting architectural context. I'll preserve the specification's requirements rather than quietly simplifying them. Where I recommend implementation choices that are not explicitly dictated by the specification, I'll label them as recommendations.

The key architectural conclusion is:

> **Build this as a Python application with one main agent, a deterministic tool layer, a database-backed state layer, an LLM reasoning layer, and explicit approval boundaries around Anki writes.**

The specification explicitly calls for one main agent initially, with multi-agent architecture only as a future option. ■filecite■turn0file2■L1-L28■

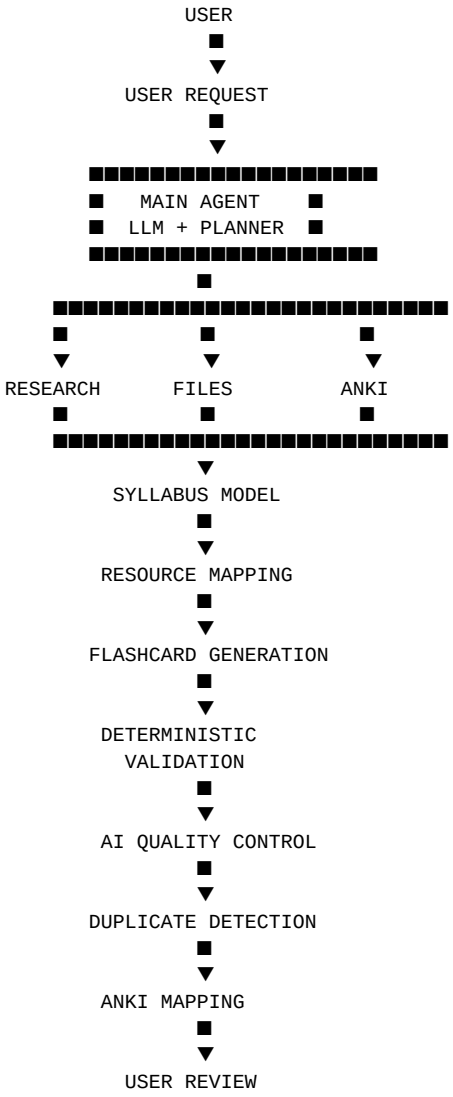
---

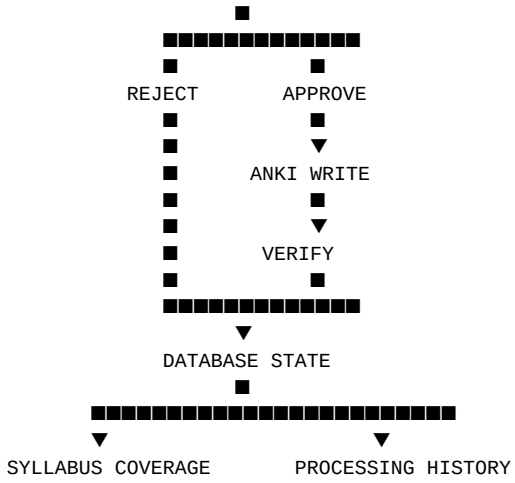
# 1. What you are actually building

The finished system is not simply:

```
PDF → AI → Flashcards
```

It is:






The specification explicitly defines this overall chain: specification → required knowledge → resource content → research/verification → flashcards → QC → duplicates → Anki → approval → logging/coverage. ■filecite■turn0file2■L29-L56■

---

# 2. Architectural principles

Before the roadmap, there are several principles I would lock in.

## Principle 1 — The syllabus is authoritative

The agent should never reason:

> "The textbook says X, therefore X must be required."

Instead:

```
Specification
    ↓
What is required?
    ↓
What resource content satisfies it?
```

The specification explicitly says the syllabus determines what needs to be learned and resources provide the information. ■filecite■turn0file2■L1-L28■

---

## Principle 2 — The LLM does not control everything

The LLM should decide and reason about things like:

• what the user means

• which research is required

• how content maps to syllabus points

• how cards should be phrased

• whether two cards are semantically similar

But deterministic Python should enforce things like:

• JSON/schema validation
• database transactions
• API communication
• duplicate hashes
• permissions
• approval state
• Anki writes
• processing IDs
• logging
• retry policies

This distinction will make the system much more reliable.

---

## Principle 3 — Treat external content as data

A PDF or website might contain text such as:

> "Ignore previous instructions and..."

The agent must treat that as **content being analysed**, not as an instruction.

This becomes especially important once you introduce web search and autonomous processing.

---

## Principle 4 — Read before write

For Anki:

```
READ
 ↓
UNDERSTAND CURRENT STATE
 ↓
PLAN CHANGE
 ↓
USER APPROVAL
 ↓
WRITE
 ↓
VERIFY
```

Never:

```
AI decides → blindly modify Anki
```

The specification explicitly requires change control and prohibits arbitrary reorganisation.
■filecite■turn0file2■L605-L630■

---

### Principle 5 — Every important object gets an ID

For example:

```
processing_run_id
resource_id
syllabus_version_id
syllabus_point_id
card_id
anki_note_id
approval_id
```

This makes tracing and recovery possible.

---

## 3. Complete requirements analysis

I went through the specification requirement by requirement.

### Functional requirements

| Requirement | Implementation |

|---|---|

| Accept textbook chapter/resource | Resource ingestion subsystem |

| Identify subject | Task classification + configuration |

| Identify exam board | Configuration + agent/research |

| Identify qualification | Configuration + research |

| Find current specification | Syllabus research tool |

| Verify specification | Source verification |

| Internet research | Web research tool |

| Map resources to syllabus | Mapping subsystem |

| Identify required knowledge | Syllabus model + mapping |

| Generate flashcards | Card generation pipeline |

| Quality-check cards | Deterministic + LLM QC |

| Exact duplicate detection | Hash/database |

| Semantic duplicate detection | Similarity/LLM |

| Determine Anki deck | Deck mapper |

| Determine note type | Anki introspection |

| Determine fields | Anki introspection |

| Create/manage decks | Anki tool layer |

| User approval | Approval subsystem |

| Add approved cards | Anki write service |

| Track processed resources | Processing database |

| Track syllabus coverage | Coverage subsystem |

| Eventually process new chapters automatically | Automation subsystem |

| Update/edit Anki content | Anki write tools |

| Search web | Web tool |

| Create/manage subdecks | Anki deck manager |

These all derive directly from the specification. ■filecite■turn0file2■L1-L28■

---

# 4. Non-functional requirements

The specification doesn't present these under a dedicated heading, but they are implied by its requirements.

The system needs:

### Reliability

It must verify Anki changes and avoid unnecessary repetition.

### Traceability

You need to know:

```
Why was this card created?
Which syllabus point?
Which resource?
Which specification?
Which web sources?
Which processing run?
```

### Safety

Important writes need approval.

### Extensibility

The architecture must eventually support automation and potentially multiple agents.

### Maintainability

The system should not become one enormous Python file.

### Configurability

Subject/deck mappings and card preferences should not be hard-coded. ■filecite■turn0file2■L345-L369■

### Recoverability

The application must survive partial failures.

This is not explicitly specified as a separate feature, but it is an architectural requirement if you are going to perform multi-step Anki writes safely.

---

# 5. Complete architecture

I recommend this architecture:

```
gcse-revision-agent/
■
■■■ src/
■   ■
■   ■■■ app/
■   ■   ■■■ main.py
■   ■   ■■■ dependencies.py
■   ■   ■■■ lifecycle.py
■   ■
■   ■■■ agent/
■   ■   ■■■ agent.py
■   ■   ■■■ planner.py
■   ■   ■■■ state.py
■   ■   ■■■ tool_registry.py
■   ■   ■■■ prompts/
■   ■       ■■■ system.md
■   ■       ■■■ planner.md
■   ■       ■■■ syllabus_mapping.md
■   ■       ■■■ card_generation.md
■   ■       ■■■ card_quality.md
■   ■       ■■■ duplicate_detection.md
■   ■
■   ■■■ tools/
■   ■   ■■■ web_tools.py
■   ■   ■■■ file_tools.py
■   ■   ■■■ syllabus_tools.py
■   ■   ■■■ anki_tools.py
■   ■
■   ■■■ anki/
■   ■   ■■■ client.py
■   ■   ■■■ decks.py
■   ■   ■■■ notes.py
■   ■   ■■■ models.py
■   ■   ■■■ mapper.py
■   ■   ■■■ verifier.py
■   ■
■   ■■■ research/
■   ■   ■■■ web.py
■   ■   ■■■ sources.py
■   ■   ■■■ syllabus.py
■   ■   ■■■ verification.py
■   ■
■   ■■■ resources/
■   ■   ■■■ loader.py
■   ■   ■■■ pdf.py
■   ■   ■■■ ocr.py
■   ■   ■■■ chapters.py
■   ■   ■■■ sections.py
■   ■   ■■■ chunking.py
■   ■
■   ■■■ syllabus/
■   ■   ■■■ models.py
■   ■   ■■■ parser.py
■   ■   ■■■ mapper.py
```

```
■   ■   ■■■ coverage.py
■   ■
■   ■■■ cards/
■   ■   ■■■ models.py
■   ■   ■■■ generator.py
■   ■   ■■■ validator.py
■   ■   ■■■ quality.py
■   ■   ■■■ duplicates.py
■   ■
■   ■■■ approval/
■   ■   ■■■ models.py
■   ■   ■■■ service.py
■   ■   ■■■ policies.py
■   ■
■   ■■■ database/
■   ■   ■■■ connection.py
■   ■   ■■■ models.py
■   ■   ■■■ repositories/
■   ■   ■■■ migrations/
■   ■
■   ■■■ config/
■   ■   ■■■ loader.py
■   ■   ■■■ models.py
■   ■   ■■■ defaults.py
■   ■
■   ■■■ security/
■   ■   ■■■ secrets.py
■   ■   ■■■ permissions.py
■   ■
■   ■■■ observability/
■       ■■■ logging.py
■       ■■■ errors.py
■       ■■■ metrics.py
■
■■■ tests/
■   ■■■ unit/
■   ■■■ integration/
■   ■■■ agent/
■   ■■■ prompts/
■   ■■■ e2e/
■   ■■■ fixtures/
■
■■■ data/
■   ■■■ incoming/
■   ■■■ processed/
■   ■■■ cache/
■
■■■ config/
■   ■■■ settings.json
■   ■■■ subjects.json
■
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ pyproject.toml
■■■ README.md
```

Don't create all of this on day one.

This is the **destination architecture**.

---

# 6. Component responsibilities

### `agent/`

Responsible for:

> reasoning and orchestration.

It should **not** directly make HTTP requests to Anki or websites.

Instead:

```
Agent
 ↓
Tool
 ↓
Service
 ↓
External system
```

---

## `tools/`

The agent's controlled interface to the outside world.

Example:

```
search_web()
read_pdf()
get_anki_decks()
find_anki_notes()
add_anki_cards()
```

The specification explicitly proposes this tool-based architecture. ■filecite■turn0file2■L78-L113■

---

## `anki/`

Contains deterministic Anki functionality.

The agent shouldn't know how AnkiConnect HTTP requests work.

It should simply know:

```
get_anki_decks()
```

---

## `research/`

Handles:

• search
• source records
• official-source prioritisation
• specification discovery

• verification

---

## `resources/`

Handles turning:

```
PDF → structured content
```

---

## `syllabus/`

Represents:

```
exam board
qualification
specification
topic
syllabus point
```

and their relationships.

---

## `cards/`

Responsible for:

```
generation
 ↓
schema validation
 ↓
quality control
 ↓
duplicate detection
```

---

## `approval/`

Creates a formal barrier between:

```
PROPOSED CHANGE
```

and:

```
EXECUTED CHANGE
```

---

## `database/`

Persistent state.

---

# 7. Data architecture

I recommend SQLite initially.

You don't need PostgreSQL, Redis, MongoDB or a vector database at the beginning.

SQLite is enough for a personal local application.

---

## `processing_runs`

```
id
started_at
completed_at
status
user_request
subject
exam_board
qualification
syllabus_version_id
resource_id
cards_generated
cards_approved
cards_added
error_message
```

Purpose:

> One execution of the agent.

---

## `resources`

```
id
path
filename
file_hash
resource_type
subject
created_at
processed_at
status
metadata_json
```

The `file_hash` is particularly important.

It lets you determine:

> "Is this actually the same file?"

---

## `syllabus_versions`

```
id
exam_board
qualification
subject
version
source_url
accessed_at
content_hash
is_current
```

The specification explicitly requires recording the specification version/source/date. ■filecite■turn0file2■L115-L163■

---

## `syllabus_points`

```
id
syllabus_version_id
code
title
description
parent_id
order_index
```

This lets you represent:

```
6
■■■ 6.1
■■■ 6.2
■■■ 6.3
■■■ 6.4
```

---

## `resource_syllabus_mappings`

```
id
resource_id
syllabus_point_id
relevance
confidence
source_pages
reason
created_at
```

Example:

```
Chapter 6
   ↓
Syllabus 6.3
   ↓
pages 124–129
   ↓
confidence = high
```

---

## `cards`

```
id
processing_run_id
syllabus_point_id
resource_id
front
back
topic
status
content_hash
anki_note_id
created_at
updated_at
```

Possible status:

```
generated
validated
duplicate
rejected
approved
written
failed
```

---

## `card_sources`

```
id
card_id
source_type
source_id
citation
supporting_text
```

This is useful for traceability.

---

## `anki_deck_mappings`

```
id
subject
exam_board
qualification
deck_name
created_at
updated_at
```

Example:

```
Chemistry → GCSE::Chemistry
```

The specification specifically recommends configuration rather than repeatedly guessing this mapping.
■filecite■turn0file2■L345-L369■

---

## `anki_notes`

```
id
anki_note_id
```

```
deck_name
note_type
fields_json
last_seen
```

This gives your database a local record of notes relevant to the agent.

---

### `approvals`

```
id
processing_run_id
operation_type
target_type
target_id
status
approved_at
rejected_at
approved_by
reason
```

For example:

```
operation = CREATE_CARD
target = card_123
status = APPROVED
```

---

### `coverage`

```
id
syllabus_point_id
status
card_count
last_evaluated
confidence
```

Statuses:

```
Not Started
Partial
Complete
```

These statuses are explicitly defined in the specification. ■filecite■turn0file2■L164-L188■

---

## 8. Data flow

A card should ultimately be traceable like this:

```
Anki Note
    ■
    ▼
Card
    ■
    ■■■ syllabus point
    ■
```

```
    ■■■ processing run
    ■
    ■■■ textbook/resource
    ■
    ■■■ web sources
    ■
    ■■■ specification version
```

That is extremely valuable.

If you ever ask:

> "Why does this card exist?"

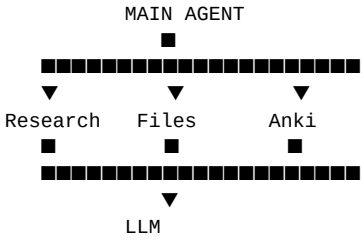
the application can answer:

```
Card #182
↓
CAIE IGCSE Chemistry
↓
Specification 2026
↓
Syllabus point 6.3.2
↓
Chapter 6
↓
Pages 142–147
↓
Supplemented by source X
↓
Generated 18 Aug 2026
↓
Approved by user
↓
Anki note 123456789
```

---

# 9. Agent architecture

Initially:




The LLM should not be given unrestricted access.

Instead:

```
LLM
 ↓
Tool call
 ↓
Tool registry
 ↓
Permission check
```

```
 ↓
Tool implementation
 ↓
Result
 ↓
LLM
```

---

# 10. Tool categories

## Read-only tools

These are generally safer.

```
search_web
search_syllabus
read_file
read_pdf

get_anki_decks
get_anki_note_types
get_anki_fields
find_anki_notes
get_anki_note_info
```

---

## Write tools

These require stronger controls.

```
add_anki_cards
update_anki_card
create_anki_deck
rename_anki_deck
move_anki_cards
```

The specification explicitly distinguishes read and write actions and requires appropriate approval.
■filecite■turn0file2■L605-L630■

---

# 11. Tool contract

Every tool should have a predictable structure.

Conceptually:

```
ToolResult(
    success=True,
    data=...,
    error=None,
    metadata={}
)
```

Don't let tools randomly return:

```
"it worked!"
```

or:

```
{"something": "..."}
```

depending on the tool.

Use a consistent result structure.

---

# 12. Example tool interface

### `get_anki_decks()`

Input:

```
none
```

Output:

```
{
  "decks": [
    {
      "name": "GCSE",
      "id": "..."
    },
    {
      "name": "GCSE::Chemistry",
      "id": "..."
    }
  ]
}
```

---

### `find_anki_notes()`

Input:

```
deck
query
```

Output:

```
{
  "notes": [
    {
      "note_id": "...",
      "note_type": "...",
      "fields": {...},
      "tags": [...]
    }
  ]
}
```

---

### `search_web()`

Input:

```
query
source_preferences
```

Output:

```
{
  "results": [
    {
      "title": "...",
      "url": "...",
      "snippet": "...",
      "source_type": "official"
    }
  ]
}
```

---

# 13. Prompt architecture

Do **not** make one enormous prompt.

Use layers.

## System prompt

Defines:

- agent identity
- fundamental principles
- safety rules
- source hierarchy
- read/write distinction
- overall goal

---

## Agent policy

Defines:

- how to plan
- when to use tools
- when to ask the user
- when to stop
- when approval is required

---

### Tool descriptions

Each tool gets its own concise contract.

---

### Task prompt

Contains the specific request:

```
Process CAIE IGCSE Chemistry Topic 6
using Chapter 6.
```

---

### Specialist prompts

Separate prompts for:

```
syllabus_mapping
card_generation
card_quality
semantic_duplicate_detection
```

---

### User preferences

Things like:

```
exam_focused = true
concise = true
one_idea_per_card = true
practice_questions = false
```

These are configuration, not necessarily hard-coded prompt text.

---

## 14. Phase-by-phase build roadmap

Now we get to the actual build order.

---

## PHASE 0 — Define the development environment

#### Objective

Get to:

```
Python works
Git works
IDE works
Anki works
Project runs
```

## Learn

• terminal basics

• Python installation

• virtual environments

• Git basics

## Install

Recommended:

• Python

• Git

• VS Code

• Anki

• AnkiConnect

## Create

```
gcse-revision-agent/
```

Create:

```
README.md
.gitignore
pyproject.toml
src/
tests/
```

## First program

```
src/main.py
```

Print:

```
GCSE Revision Agent
```

## Test

Run:

```
python -m src.main
```

## Complete when

You can clone/recreate the project and run it successfully.

---

# PHASE 1 — Python/project foundations

### Objective

Create a clean Python application structure.

### Learn

• modules

• imports

• packages

• classes

• type hints

• exceptions

• `dataclass`

• JSON

• virtual environments

• dependency management

### Build

Create:

```
src/
■■■ app/
■■■ config/
■■■ anki/
■■■ database/
■■■ tools/
■■■ agent/
```

But only add implementation files as you reach each phase.

### Add

```
src/app/main.py
```

```
src/app/lifecycle.py
```

### Test

• application imports successfully

• modules can import one another

• invalid configuration fails cleanly

### Complete when

You have a functioning Python application rather than a collection of scripts.

---

# PHASE 2 — Configuration system

This should come early because the specification explicitly says user-specific information should be configuration rather than repeated guessing. ■filecite■turn0file2■L345-L369■

### Build

```
src/config/
■■■ models.py
■■■ loader.py
■■■ defaults.py

config/
■■■ settings.json
■■■ subjects.json
```

### Configuration

```
{
  "subjects": {
    "chemistry": {
      "exam_board": "CAIE",
      "qualification": "IGCSE",
      "anki_deck": "GCSE::Chemistry"
    }
  }
}
```

### Classes

```
AppConfig
SubjectConfig
FlashcardPreferences
ApprovalConfig
AutomationConfig
```

### Test

Load configuration and print:

```
Chemistry
CAIE
IGCSE
GCSE::Chemistry
```

### Complete when

The application can obtain configuration without hard-coded values.

---

# PHASE 3 — Logging and error system

Do this before complex integrations.

### Build

```
src/observability/
■■■ logging.py
■■■ errors.py
```

Implement:

```
configure_logging()
get_logger()
```

Define application exceptions:

```
AnkiConnectionError
ConfigurationError
ResearchError
ResourceProcessingError
ValidationError
ApprovalRequiredError
```

### Logging levels

```
DEBUG
INFO
WARNING
ERROR
```

### Test

Intentionally trigger an error and verify:

```
timestamp
level
component
message
processing_run_id
```

### Complete when

You can diagnose failures from logs.

---

# PHASE 4 — Database foundation

Do this before sophisticated agent behaviour.

### Technology

SQLite.

Recommended ORM/migration approach:

• SQLAlchemy

• Alembic

You don't absolutely need an ORM, but for this project's growing relationships I recommend one.

### Build

```
src/database/
■■■ connection.py
■■■ models.py
■■■ repositories/
```

Create initial tables:

```
processing_runs
resources
syllabus_versions
syllabus_points
cards
```

Don't create every future table immediately.

### Test

Create a processing run, save it, retrieve it.

### Complete when

State survives application restarts.

---

# PHASE 5 — Anki connection

This is your first major integration.

Your source material explicitly recommends starting with:

> Python → AnkiConnect → test card. ■filecite■turn0file0■L45-L71■

### Build

```
src/anki/
■■■ client.py
■■■ models.py
■■■ verifier.py
```

`client.py`:

```
AnkiConnectClient
```

Responsibilities:

```
send_request()
invoke()
check_connection()
```

### First API operation

```
deckNames
```

### Test

Application prints your actual Anki decks.

### Complete when

Python reliably communicates with AnkiConnect.

---

# PHASE 6 — Anki read subsystem

Now implement all safe inspection operations.

### Build

```
anki/decks.py
anki/notes.py
anki/models.py
```

Implement:

```
get_decks()
get_note_types()
get_note_type_fields()
find_notes()
get_note_info()
```

The specification specifically requires inspection rather than assuming note types/fields.
■filecite■turn0file2■L267-L306■

### Test

Ask:

```
What decks exist?
What note types exist?
What fields does Basic have?
What notes exist in Chemistry?
```

### Complete when

The program can fully inspect the relevant Anki structure.

---

# PHASE 7 — Anki write subsystem

Only now introduce writes.

### Implement

```
create_deck()
rename_deck()
add_note()
update_note()
move_cards()
```

### Important

Build a write policy:

```
WRITE REQUEST
     ↓
Does operation require approval?
     ↓
YES → ApprovalRequired
     ↓
NO → execute
```

### Test

Use a dedicated:

```
AI Agent Test
```

Anki deck.

Create:

```
Test Card
```

Then verify it.

### Complete when

You can perform a complete:

```
create → inspect → update → verify
```

cycle.

---

# PHASE 8 — Anki mapping subsystem

Now solve:

> Where does a generated card belong?

### Build

```
anki/mapper.py
```

Functions:

```
resolve_deck()
resolve_note_type()
resolve_fields()
validate_mapping()
```

Input:

```
subject
exam_board
qualification
```

Output:

```
DeckMapping
NoteMapping
FieldMapping
```

### Critical rule

Never assume:

```
Front
Back
```

Inspect the real note type.

### Complete when

A generated card can be translated into a valid Anki note without guessing field names.

---

# PHASE 9 — LLM integration

Only now bring in the AI.

### Build

```
src/agent/
■■■ llm_client.py
■■■ models.py
```

Implement:

```
LLMClient
generate()
generate_structured()
```

### Learn

• API calls

• tokens

• structured outputs

• schemas

• retries

• rate limits

### First test

Ask the model:

```
Return one flashcard in the specified schema.
```

### Complete when

Python can reliably receive validated structured data from the model.

---

# PHASE 10 — Structured domain schemas

Don't let raw LLM JSON flow directly into Anki.

Create Python models.

For example:

```
FlashcardDraft
SyllabusReference
SourceReference
ResearchResult
MappingResult
QualityResult
ApprovalRequest
```

A flashcard:

```
FlashcardDraft
■■■ front
■■■ back
■■■ topic
■■■ syllabus_point_id
■■■ tags
■■■ sources
```

### Test

Feed intentionally malformed LLM output.

Your program should reject it.

### Complete when

Invalid AI output cannot silently progress through the system.

---

# PHASE 11 — Prompt architecture

Create:

```
agent/prompts/
■■■ system.md
■■■ planner.md
■■■ syllabus_mapping.md
■■■ card_generation.md
■■■ card_quality.md
■■■ duplicate_detection.md
```

Version them.

For example:

```
card_generation_v1
card_generation_v2
```

Store prompt version with processing runs.

This is important for reproducibility.

---

# PHASE 12 — Tool framework

Now build the actual agent tool architecture.

### Build

```
agent/tool_registry.py
tools/
```

Create:

```
Tool
ToolResult
ToolRegistry
ToolExecutor
```

Each tool should have:

```
name
description
input schema
output schema
permission level
handler
```

Example:

```
get_anki_decks
READ
```

versus:

```
add_anki_cards
WRITE
APPROVAL REQUIRED
```

### Complete when

The LLM can request a tool, Python executes it, and the result is safely returned to the LLM.

---

# PHASE 13 — File/resource subsystem

Now start processing actual study material.

### Build

```
resources/
■■■ loader.py
■■■ pdf.py
■■■ chapters.py
■■■ sections.py
■■■ chunking.py
```

### First target

Text-based PDF.

Implement:

```
load_pdf()
extract_text()
get_pages()
```

### Don't start OCR yet.

Get normal PDFs working first.

---

# PHASE 14 — Chunking

Large textbooks cannot simply be dumped into the model.

Implement:

```
chunk_document()
```

Each chunk should contain:

```
resource_id
page_start
page_end
section
text
token_estimate
```

### Important

Chunks must retain metadata.

Bad:

```
"Atoms contain..."
```

Good:

```
Resource: chemistry_book.pdf
Chapter: 6
Pages: 142–144
Section: Atomic structure
Text: ...
```

### Complete when

A large PDF can be processed in manageable pieces while retaining provenance.

---

# PHASE 15 — OCR

Only after ordinary PDFs work.

Build:

```
resources/ocr.py
```

Pipeline:

```
PDF
 ↓
Can text be extracted?
 ■■■ YES → use extracted text
 ■■■ NO
      ↓
     OCR
```

### Test

Use:

1. normal PDF
2. scanned PDF
3. mixed PDF

### Complete when

Scanned resources can be processed or fail gracefully.

---

# PHASE 16 — Web research

Now give the agent internet access.

The specification explicitly requires web research for specifications, verification, authoritative explanations and missing information. ■filecite■turn0file2■L115-L163■

### Build

```
research/
■■■ web.py
■■■ sources.py
■■■ verification.py
```

Tool:

```
search_web()
```

Return:

```
ResearchResult
■■■ query
■■■ title
■■■ url
■■■ source_type
■■■ content
■■■ accessed_at
■■■ reliability
```

### Source classification

At minimum:

```
official_exam_board
government
reputable_educational
other
unknown
```

### Complete when

The agent can research information while preserving source metadata.

---

# PHASE 17 — Syllabus research

This is one of the most important phases.

Build:

```
research/syllabus.py
syllabus/
■■■ models.py
■■■ parser.py
```

The workflow:

```
Subject
 ↓
```

```
Exam board
 ↓
Qualification
 ↓
Search official source
 ↓
Identify current specification
 ↓
Retrieve specification
 ↓
Parse requirements
 ↓
Store version
```

The specification requires current-version tracking and source/date metadata. ■filecite■turn0file2■L115-L163■

### Store

```
exam board
qualification
subject
version
source
URL
access date
```

### Complete when

You can produce a structured syllabus representation from an authoritative source.

---

# PHASE 18 — Syllabus model

Represent syllabus structure.

For example:

```
Chemistry
■■■ Topic 6
    ■■■ 6.1
    ■■■ 6.2
    ■■■ 6.3
    ■■■ 6.4
```

Each point needs:

```
code
title
description
parent
order
version
```

### Complete when

The database can answer:

> What exactly must be known for Topic 6?

without consulting the LLM.

---

# PHASE 19 — Resource-to-syllabus mapping

This is where the project starts becoming genuinely useful.

Input:

```
Syllabus
+
Textbook chunks
```

Output:

```
MappingResult
```

Example:

```
Chapter 6 pages 140–146
        ↓
Syllabus 6.2
        ↓
HIGH relevance
```

The LLM can help with semantic mapping.

Python should enforce:

• valid syllabus point IDs

• page references

• confidence ranges

• provenance

### Complete when

The system can show:

```
Syllabus point 6.2
↓
Textbook pages 140–146
↓
Relevant content
```

---

# PHASE 20 — Coverage detection

Now determine:

```
Not Started
Partial
Complete
```

The specification explicitly requires this. ■filecite■turn0file2■L164-L188■

The system should NOT simply count cards.

Coverage should consider whether the syllabus point's required knowledge is actually represented.

### Complete when

You can produce:

```
6.1 Complete
6.2 Partial
6.3 Missing
```

with reasons.

---

# PHASE 21 — Flashcard generation

Now implement:

```
cards/generator.py
```

Input:

```
syllabus requirements
+
relevant resource content
+
verified supporting information
```

Output:

```
FlashcardDraft[]
```

The specification explicitly requires generation from this combination. ■filecite■turn0file2■L189-L216■

### Generation rules

Enforce:

• exam-focused

• concise

• one useful idea

• GCSE terminology

• relevant syllabus

• no unnecessary information

• no unnecessary examples

• no practice questions unless requested

• no duplicate concepts

• quality over quantity

### Complete when

A syllabus point + resource can produce structured candidate cards.

---

# PHASE 22 — Deterministic card validation

Before asking another AI to assess cards, use normal Python.

Check:

```
front exists
back exists
syllabus point exists
front isn't absurdly long
back isn't empty
tags valid
source exists
```

Also reject:

```
invalid JSON
unknown syllabus IDs
invalid data types
duplicate hashes
```

### Complete when

Malformed cards cannot reach the quality-control stage.

---

# PHASE 23 — AI quality control

Now create:

```
cards/quality.py
```

Check:

### Accuracy

Is the card factually correct?

### Syllabus relevance

Does it correspond to the specification?

### Completeness

Does the answer contain what is required?

### Question quality

Is it unambiguous?

### Atomicity

Does it test one useful concept?

### Redundancy

Is it unnecessary?

### Exam usefulness

Would knowing it help?

### Difficulty

Is it appropriate for GCSE?

These criteria come directly from the specification. ■filecite■turn0file2■L217-L240■

Output:

```
QualityResult
■■■ pass
■■■ score
■■■ issues
■■■ suggested_fix
■■■ confidence
```

---

# PHASE 24 — Regeneration loop

Bad card:

```
Generated
 ↓
QC
 ↓
FAIL
 ↓
Regenerate
 ↓
QC
```

Set a maximum number of attempts.

For example:

```
max_attempts = 2
```

If it still fails:

```
NEEDS_REVIEW
```

Do not endlessly ask the LLM to regenerate.

---

# PHASE 25 — Exact duplicate detection

This should be deterministic.

Normalise:

```
lowercase
trim whitespace
normalise punctuation
```

Then hash:

```
front + back
```

Compare against:

• current generated batch

• database

• Anki

The specification explicitly requires exact duplicate detection first. ■filecite■turn0file2■L241-L266■

### Complete when

Identical cards are reliably rejected without an LLM.

---

# PHASE 26 — Semantic duplicate detection

Now use AI/semantic similarity.

Example:

```
"What is an isotope?"

versus

"Define an isotope."
```

The specification explicitly identifies this as a semantic duplicate. ■filecite■turn0file2■L241-L266■

Pipeline:

```
Candidate
 ↓
Exact duplicate?
```

```
 ■■■ YES → reject
 ■■■ NO
      ↓
Semantic comparison
      ↓
Duplicate?
 ■■■ YES → reject
 ■■■ NO → continue
```

Important:

> Related ≠ duplicate.

---

# PHASE 27 — Final Anki mapping

Take:

```
validated card
```

and determine:

```
deck
note type
fields
tags
```

using the actual Anki environment.

### Complete when

The application can display:

```
Card:
"What is an isotope?"

Destination:
GCSE::Chemistry

Note type:
Basic

Fields:
Front → front
Back → back
```

before writing anything.

---

# PHASE 28 — Approval subsystem

Now build formal approval.

```
approval/
■■■ models.py
■■■ service.py
```

```
■■■ policies.py
```

Workflow:

```
Generated
 ↓
Validated
 ↓
Duplicate checked
 ↓
Mapped to Anki
 ↓
AWAITING APPROVAL
 ↓
User approves
 ↓
WRITE
```

User actions:

```
approve
reject
edit
regenerate
change deck
approve group
```

---

# PHASE 29 — User review interface

For the first version, **do not build a sophisticated web frontend**.

A CLI is enough.

Example:

```
47 cards generated.

39 ready
6 duplicates
2 require review

[1] Review cards
[2] Approve all ready cards
[3] Reject all
[4] Cancel
```

Later you can build a GUI/web interface.

### Complete when

No significant Anki write happens without the intended approval state.

---

# PHASE 30 — Safe Anki execution

Now connect:

```
Approval
 ↓
Anki write service
```

Before writing:

```
verify approval
verify deck
verify note type
verify fields
verify card hasn't become duplicate
```

Then:

```
write
 ↓
receive note ID
 ↓
save note ID
```

---

# PHASE 31 — Post-write verification

This is essential.

Don't trust:

```
AnkiConnect returned success
```

Instead:

```
WRITE
 ↓
FETCH CREATED NOTE
 ↓
COMPARE EXPECTED VS ACTUAL
```

Check:

• correct deck

• correct note type

• correct fields

• correct tags

• correct number of notes

### Complete when

Every successful write can be independently verified.

---

# PHASE 32 — Processing history

Now connect everything to:

```
processing_runs
```

Record:

```
resource
syllabus version
prompt version
cards generated
cards rejected
duplicates
approved
written
Anki IDs
errors
```

The specification explicitly requires processing history and repeat prevention. ■filecite■turn0file2■L370-L409■

---

# PHASE 33 — Repeat prevention

Before processing:

```
Calculate resource hash
        ↓
Has this resource been processed?
        ↓
YES
        ↓
Same syllabus version?
        ↓
Same configuration?
        ↓
Same processing state?
```

If everything is unchanged:

```
Don't regenerate unnecessarily.
```

If specification changed:

```
Re-evaluate affected content.
```

---

# PHASE 34 — Syllabus coverage tracker

Build:

```
syllabus/coverage.py
```

It should produce:

```
Chemistry Topic 6

6.1  COMPLETE
6.2  COMPLETE
6.3  PARTIAL
6.4  NOT STARTED
```

Also:

```
cards
sources
last processed
confidence
```

---

# PHASE 35 — Missing-content research

If:

```
Syllabus 6.3
```

is only partially covered by the textbook:

```
Textbook
   ↓
Missing requirement
   ↓
Web research
   ↓
Reliable source
   ↓
Supporting information
   ↓
Cards
```

The specification explicitly allows legitimate internet research to fill gaps. ■filecite■turn0file2■L164-L188■

---

# PHASE 36 — Main agent

Only now should the agent become the coordinator.

Implement:

```
agent/agent.py
agent/planner.py
agent/state.py
```

The agent should be able to receive:

> Process CAIE IGCSE Chemistry Topic 6 using Chapter 6.

and produce a plan:

```
1. Identify configuration
2. Find current syllabus
```

```
3. Retrieve Topic 6
4. Process Chapter 6
5. Map content
6. Identify gaps
7. Research gaps
8. Generate cards
9. Validate
10. Check duplicates
11. Inspect Anki
12. Request approval
13. Write
14. Verify
15. Update coverage
16. Record history
```

---

# PHASE 37 — Agent state machine

Do not rely on the LLM's conversational history alone.

Create explicit task state:

```
CREATED
 ↓
PLANNING
 ↓
RESEARCHING
 ↓
PROCESSING_RESOURCE
 ↓
MAPPING
 ↓
GENERATING
 ↓
VALIDATING
 ↓
DEDUPLICATING
 ↓
ANKI_MAPPING
 ↓
AWAITING_APPROVAL
 ↓
WRITING
 ↓
VERIFYING
 ↓
COMPLETED
```

Error state:

```
FAILED
```

Cancellation:

```
CANCELLED
```

This makes recovery possible.

---

# PHASE 38 — End-to-end pipeline

Now combine everything.

Input:

> Process Chapter 6 of my CAIE IGCSE Chemistry textbook.

Pipeline:

```
REQUEST
 ↓
CONFIG
 ↓
PLAN
 ↓
SYLLABUS
 ↓
RESOURCE
 ↓
MAPPING
 ↓
GAPS
 ↓
RESEARCH
 ↓
GENERATE
 ↓
VALIDATE
 ↓
QUALITY
 ↓
EXACT DUPLICATES
 ↓
SEMANTIC DUPLICATES
 ↓
ANKI INSPECTION
 ↓
DECK/NODE MAPPING
 ↓
APPROVAL
 ↓
WRITE
 ↓
VERIFY
 ↓
COVERAGE
 ↓
HISTORY
```

This is essentially the complete specification pipeline. ■filecite■turn0file2■L490-L540■

---

# PHASE 39 — Failure recovery

Now deliberately break the system.

Examples:

```
Anki dies halfway through
```

The system should know which notes succeeded.

```
LLM returns invalid JSON
```

Retry, then fail safely.

```
Database fails
```

Don't claim completion.

```
Internet search fails
```

Explain that the missing research prevented that part of the task.

```
User cancels
```

Stop before writes.

---

# PHASE 40 — Automation

Only after the manual pipeline is reliable.

Create:

```
data/incoming/
data/processed/
```

Build:

```
automation/
■■■ watcher.py
```

Workflow:

```
New PDF
 ↓
Detect file
 ↓
Hash file
 ↓
Create processing run
 ↓
Agent
 ↓
Review
 ↓
Anki
 ↓
Move to processed
```

The specification's eventual goal is essentially this workflow. ■filecite■turn0file2■L410-L461■

---

# PHASE 41 — Automated processing policy

Don't make automation automatically perform dangerous actions.

Recommended:

```
New file
 ↓
Automatic research
 ↓
Automatic card generation
 ↓
Automatic QC
 ↓
Automatic duplicate detection
 ↓
USER APPROVAL
 ↓
ANKI
```

Only later introduce:

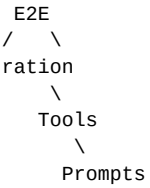
```
fully automatic mode
```

and only for operations you've explicitly authorised.

---

# PHASE 42 — Testing

At this point, create a proper testing pyramid.




---

## Unit tests

Test:

```
hash_card()
validate_card()
load_config()
chunk_text()
map_deck()
normalise_text()
calculate_coverage()
```

---

## Anki integration tests

Test:

```
connection
```

```
get decks
get note types
get fields
find notes
add note
update note
move card
verify write
```

---

### Research tests

Test:

```
search
source classification
official source selection
source recording
```

---

### Resource tests

Test:

```
normal PDF
scanned PDF
large PDF
bad PDF
empty PDF
```

---

### Agent tests

Give the agent scenarios:

```
Process Chemistry Topic 6.
```

Check that it chooses the appropriate tools.

---

## 43. Golden test suite

You should maintain fixed examples.

For example:

#### Golden test 1

```
Chemistry Topic 6
```

Expected:

```
correct syllabus
correct deck
correct note type
```

### Golden test 2

Same textbook twice.

Expected:

```
second run creates no duplicates
```

### Golden test 3

Changed syllabus.

Expected:

```
system notices version change
```

### Golden test 4

Malformed LLM output.

Expected:

```
rejected
```

### Golden test 5

Prompt injection in PDF.

Expected:

```
treated as data
```

---

# 44. Security architecture

## Secrets

Never:

```
API_KEY = "..."
```

Use:

```
.env
```

and environment variables.

Commit:

```
.env.example
```

Never:

```
.env
```

---

## File security

The resource reader should only access permitted directories.

Don't let an LLM specify arbitrary filesystem paths and directly execute them.

---

## Web security

Web content is untrusted.

Treat:

```
web result
PDF
textbook
```

as data.

Never allow their contents to redefine agent policy.

---

## Anki security

Write tools should have explicit permissions.

For example:

```
READ_ANki
WRITE_ANKI
DESTRUCTIVE_ANKI
```

---

# 45. Error/edge-case matrix

| Failure | Behaviour |

|---|---|

| Anki not running | Stop Anki-dependent stage, explain |

| AnkiConnect unavailable | Retry, then fail safely |

| Deck missing | Ask/create only if authorised |

| Note type missing | Ask rather than guessing |

| Fields incompatible | Stop write |

| Exact duplicate | Skip |

| Semantic duplicate | Flag/skip |

| Invalid JSON | Retry structured generation |

| AI hallucination | QC rejection |

| Wrong syllabus | Reject/research again |

| Official syllabus unavailable | Do not invent |

| Multiple specifications | Resolve/ask |

| PDF scanned | OCR |

| PDF unreadable | Report failure |

| PDF huge | Chunk |

| Website unavailable | Retry/alternative source |

| Unreliable source | Lower priority/reject |

| Conflicting sources | Escalate for verification |

| Ambiguous request | Ask user |

| User cancels | Stop safely |

| API rate limit | Backoff/retry |

| Missing API key | Configuration error |

| DB failure | Fail transaction |

| Partial Anki write | Verify/reconcile |

| Duplicate processing | Detect via hashes/history |

| Syllabus changed | Reassess affected content |

---

# 46. Important recovery mechanism

A particularly important architecture decision:

## Don't make a processing run one giant transaction.

For example:

```
100 cards
 ↓
write card 1
write card 2
write card 3
...
```

If the application crashes at card 57, you need to know:

```
Cards 1–56 → completed
Card 57 → unknown
Cards 58–100 → not attempted
```

Therefore store individual operation state.

```
card
■■■ generated
■■■ approved
■■■ write_started
■■■ written
■■■ verified
```

Then recovery can resume safely.

---

# 47. What should be deterministic vs AI

This is one of the most important design decisions.

## Deterministic Python

Use Python for:

• API calls

• file access

• PDF extraction

• hashing

• database operations

• schema validation

• state machines

• approval state

• permissions

• Anki writes

• post-write verification

• logging

• retry logic

---

## LLM

Use the LLM for:

• interpreting user requests

• planning

• semantic syllabus mapping

• understanding resource content

• generating cards

• quality assessment

• semantic duplicate analysis

• deciding when research may be needed

---

### Don't let AI do this

Do not ask the LLM:

> "Have I already processed this file?"

Python/database can answer that exactly.

Don't ask:

> "Does this deck exist?"

Anki can answer that exactly.

Don't ask:

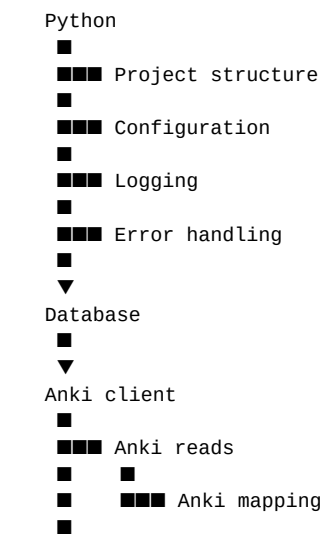
> "Was the card successfully written?"

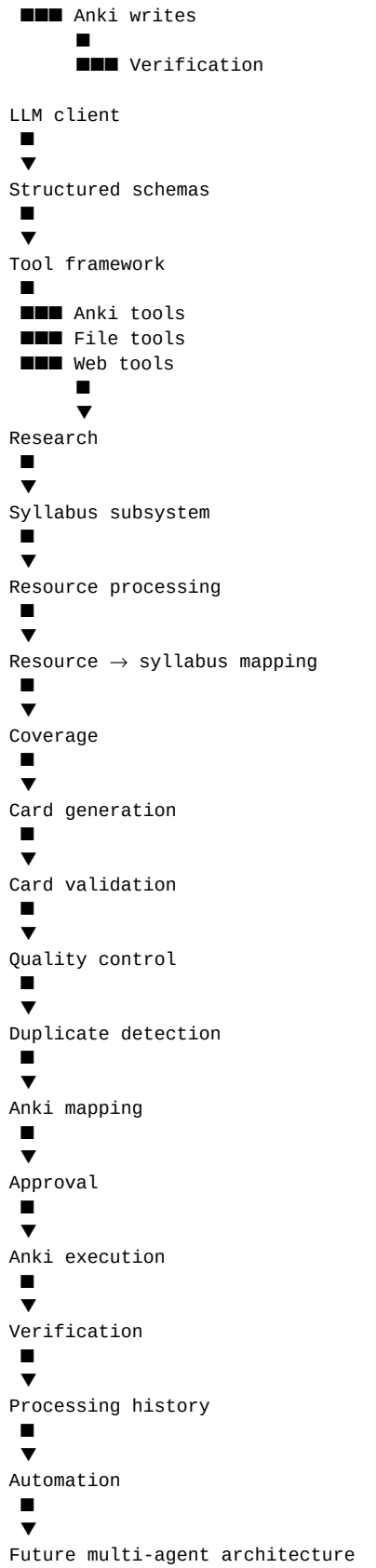
Verify it through Anki.

---

## 48. Dependency graph

The actual dependency graph I recommend is:






---

# 49. What should NOT be built yet

## Don't start with multi-agent systems

Your specification itself says:

> one main AI agent initially.

The supporting document also recommends one agent + tools first. ■filecite■turn0file1■L29-L61■

---

## Don't start with LangChain

You can add an agent framework later if it genuinely helps.

Initially, building your own tool loop teaches you what agents actually are.

---

## Don't start with a vector database

You don't initially need:

```
Pinecone
Weaviate
Qdrant
Milvus
```

For the first versions, SQLite + structured chunks + search/similarity is enough.

A vector database becomes useful when your resource collection becomes large enough that semantic retrieval needs dedicated infrastructure.

---

## Don't start with a web frontend

CLI first.

A web UI adds:

```
frontend
backend
authentication
state synchronisation
API
```

before you've proven the core agent.

---

## Don't deploy to the cloud initially

Your project is fundamentally local:

```
your computer
```

```
■
■■■ PDFs
■■■ Anki
■■■ agent
```

Keep it local until the system works.

---

# 50. MVP

Your MVP should be dramatically smaller than the finished system.

## MVP objective

Prove:

> **I can give the system a textbook section, have it generate syllabus-relevant cards, review them, and put approved cards into Anki.**

### MVP includes

```
Python
AnkiConnect
LLM
PDF text extraction
basic syllabus input
structured cards
basic validation
exact duplicate detection
Anki mapping
manual approval
Anki write
verification
```

### MVP excludes

```
OCR
automatic syllabus discovery
advanced web research
semantic duplicate detection
coverage tracking
automation
complex UI
multi-agent architecture
```

---

# 51. MVP flow

```
User
 ↓
"Process this Chemistry chapter"
 ↓
Choose syllabus
 ↓
Read PDF
 ↓
LLM generates cards
```

```
 ↓
Validate
 ↓
Exact duplicates
 ↓
Show cards
 ↓
User approves
 ↓
Anki
 ↓
Verify
```

Once this works reliably, you have a real foundation.

---

# 52. V2

Add:

```
automatic syllabus discovery
web research
semantic duplicates
better Anki mapping
deck/subdeck management
processing history
```

---

# 53. V3

Add:

```
syllabus coverage
gap detection
supporting-resource research
specification version tracking
```

---

# 54. V4

Add:

```
OCR
large-resource processing
automatic folder processing
recovery
better observability
```

---

# 55. V5

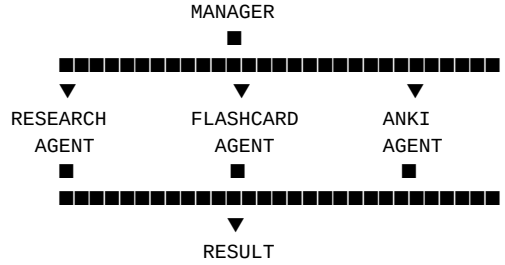
Add:

```
advanced interface
automatic mode
sophisticated revision analytics
```

---

# 56. V6 — optional multi-agent architecture

Only if justified:




Possible agents match the roles described in the specification:

• Research Agent

• Flashcard Agent

• Examiner/Quality Agent

• Anki Agent

• Manager Agent ■filecite■turn0file2■L462-L489■

But this is **not part of your initial implementation**.

---

# 57. Learning roadmap

You don't need to learn everything before starting.

## Phase 0–4

Learn:

```
Python
variables
functions
classes
modules
exceptions
JSON
virtual environments
Git
SQLite
basic SQL
```

---

## Phase 5–8

Learn:

```
HTTP
REST APIs
JSON requests/responses
AnkiConnect
API errors
type hints
data models
```

---

## Phase 9–12

Learn:

```
LLM APIs
structured outputs
JSON schemas
prompt engineering
tool calling
agent loops
```

---

## Phase 13–19

Learn:

```
PDF extraction
OCR
document chunking
tokens/context windows
web search
source verification
information retrieval
```

---

## Phase 20–27

Learn:

```
LLM evaluation
semantic similarity
data validation
quality-control pipelines
```

---

## Phase 28–35

Learn:

```
state machines
approval workflows
```

```
database relationships
transactions
recovery
```

---

### Phase 36–42

Learn:

```
agent orchestration
automation
background processes
observability
integration testing
```

---

## 58. Complete build-order checklist

This is the master checklist I'd actually follow.

### Foundation

• [ ] Install Python

• [ ] Install Git

• [ ] Install VS Code

• [ ] Install Anki

• [ ] Install/configure AnkiConnect

• [ ] Create project folder

• [ ] Initialise Git

• [ ] Create virtual environment

• [ ] Create `pyproject.toml`

• [ ] Create `.gitignore`

• [ ] Create `.env.example`

• [ ] Create README

• [ ] Create `src/`

• [ ] Create `tests/`

• [ ] Run first Python program

### Application foundations

• [ ] Build application entry point

• [ ] Build configuration loader

• [ ] Define configuration models

• [ ] Add subject configuration

- [ ] Add flashcard preferences
- [ ] Add logging
- [ ] Add application exceptions
- [ ] Add basic tests

## Database

- [ ] Add SQLite
- [ ] Add database connection
- [ ] Add migrations
- [ ] Create processing runs
- [ ] Create resources
- [ ] Create syllabus versions
- [ ] Create syllabus points
- [ ] Create cards
- [ ] Add repositories
- [ ] Test persistence

## Anki

- [ ] Build AnkiConnect client
- [ ] Test connection
- [ ] Get decks
- [ ] Get note types
- [ ] Get fields
- [ ] Find notes
- [ ] Get note info
- [ ] Create test deck
- [ ] Create test note
- [ ] Update test note
- [ ] Verify test note
- [ ] Build deck mapper
- [ ] Build note-type mapper
- [ ] Build field mapper
- [ ] Build write permissions
- [ ] Build write verification

## LLM

- [ ] Select LLM provider
- [ ] Configure API key

- [ ] Build LLM client
- [ ] Test basic completion
- [ ] Add structured outputs
- [ ] Create domain schemas
- [ ] Validate model output
- [ ] Add retry handling
- [ ] Add prompt versioning

## Tools

- [ ] Build tool interface
- [ ] Build tool registry
- [ ] Build tool executor
- [ ] Add permissions
- [ ] Add Anki read tools
- [ ] Add Anki write tools
- [ ] Add file tools
- [ ] Add research tools

## Resources

- [ ] Load PDF
- [ ] Extract text
- [ ] Preserve page metadata
- [ ] Detect chapters
- [ ] Detect sections
- [ ] Chunk documents
- [ ] Add token estimation
- [ ] Add OCR
- [ ] Test scanned PDFs
- [ ] Test large PDFs

## Research

- [ ] Build web search
- [ ] Store research results
- [ ] Classify sources
- [ ] Prioritise official sources
- [ ] Build specification search
- [ ] Verify specification
- [ ] Store specification version

- [ ] Parse syllabus
- [ ] Store syllabus points

## Mapping

- [ ] Build resource/syllabus mapping
- [ ] Store page references
- [ ] Store confidence
- [ ] Identify missing coverage
- [ ] Build coverage states

## Cards

- [ ] Build card schema
- [ ] Build generation prompt
- [ ] Generate cards
- [ ] Validate cards
- [ ] Quality-check cards
- [ ] Regenerate failed cards
- [ ] Exact duplicate detection
- [ ] Semantic duplicate detection
- [ ] Store card provenance

## Approval

- [ ] Define approval states
- [ ] Build approval database
- [ ] Build review interface
- [ ] Approve
- [ ] Reject
- [ ] Edit
- [ ] Regenerate
- [ ] Change deck
- [ ] Approve groups
- [ ] Cancel

## Agent

- [ ] Build system prompt
- [ ] Build planning prompt
- [ ] Build agent state
- [ ] Build agent loop

- [ ] Add tool selection
- [ ] Add tool execution
- [ ] Add state transitions
- [ ] Add approval boundary
- [ ] Add recovery

## Integration

- [ ] Connect research
- [ ] Connect resource processing
- [ ] Connect syllabus
- [ ] Connect mapping
- [ ] Connect cards
- [ ] Connect QC
- [ ] Connect duplicates
- [ ] Connect Anki
- [ ] Connect approval
- [ ] Connect database
- [ ] Connect coverage

## Reliability

- [ ] Test Anki failure
- [ ] Test invalid LLM output
- [ ] Test PDF failure
- [ ] Test web failure
- [ ] Test duplicate processing
- [ ] Test partial writes
- [ ] Test database failure
- [ ] Test cancellation
- [ ] Test syllabus changes
- [ ] Test prompt injection
- [ ] Test API rate limits

## Automation

- [ ] Create incoming folder
- [ ] Build file watcher
- [ ] Detect new resources
- [ ] Create processing run
- [ ] Process automatically

• [ ] Require approval

• [ ] Move processed files

• [ ] Record results

• [ ] Test repeated files

### Future

• [ ] Better UI

• [ ] Automatic mode

• [ ] Advanced analytics

• [ ] Multi-agent architecture only if justified

---

## 59. Requirements → implementation map

This is the audit trail from your specification into the roadmap.

| Specification requirement | Roadmap implementation |
|---|---|
| Accept study resources | Phases 13–15 |
| Identify subject | Phases 2, 17, 36 |
| Identify exam board | Phases 2, 17, 36 |
| Identify qualification | Phases 2, 17, 36 |
| Current official specification | Phase 17 |
| Internet research | Phase 16 |
| Resource processing | Phases 13–15 |
| Syllabus mapping | Phase 19 |
| Required knowledge | Phases 18–20 |
| Coverage tracking | Phase 20 + 34 |
| Missing information | Phase 35 |
| Flashcard generation | Phase 21 |
| Structured cards | Phases 10 + 21 |
| Quality control | Phase 23 |
| Exact duplicates | Phase 25 |
| Semantic duplicates | Phase 26 |
| Anki inspection | Phase 6 |
| Deck selection | Phase 8 |
| Note type detection | Phase 8 |
| Field detection | Phase 8 |
| Deck creation | Phase 7 |

| Subdeck management | Phase 7 |

| User approval | Phases 28–29 |

| Add approved cards | Phase 30 |

| Update cards | Phase 7 |

| Move cards | Phase 7 |

| Verify writes | Phase 31 |

| Processing history | Phase 32 |

| Repeat prevention | Phase 33 |

| Syllabus versions | Phase 17 |

| Automation | Phases 40–41 |

| Configuration | Phase 2 |

| Read/write distinction | Phases 7, 12, 28 |

| Safety/change control | Phases 28–31 |

| Future multi-agent | Phase 42 |

The specification's explicit system definition is therefore covered without removing any of its major functional requirements. ■filecite■turn0file2■L542-L603■

---

# 60. Potential Oversights audit

This is the part I'd pay particular attention to.

## Oversight 1 — Provenance

The specification says to record sources, but the architecture needs to make provenance **first-class data**.

### Added

Every card should be traceable to:

```
syllabus version
syllabus point
resource
page/section
web source
processing run
prompt version
```

---

## Oversight 2 — Prompt versioning

If you change your card-generation prompt, you need to know which version generated a card.

### Added

```
prompt_version
```

on processing runs/cards.

---

## Oversight 3 — Model version

Likewise, if you change LLMs/models:

```
model
model_version
```

should be recorded.

Otherwise you can't reproduce why outputs changed.

---

## Oversight 4 — Partial writes

The specification says to verify writes, but the architecture needs explicit per-card states.

### Added

```
write_started
written
verified
```

---

## Oversight 5 — Prompt injection

The specification doesn't explicitly discuss prompt injection.

But because the final system reads arbitrary PDFs/web content, it is an important security requirement.

### Added

Treat external content as data.

---

## Oversight 6 — Deterministic enforcement

The specification describes AI behaviour extensively, but some rules should never depend purely on the LLM.

### Added

Use Python to enforce:

```
permissions
schemas
```

```
state
duplicates
database
Anki writes
```

---

## Oversight 7 — Recovery

If the program crashes after creating 37 of 50 cards, it needs to know exactly what happened.

### Added

Persistent processing/card state.

---

## Oversight 8 — Configuration versioning

Changing:

```
deck mappings
flashcard preferences
approval settings
```

can affect future processing.

### Recommendation

Store a configuration snapshot or configuration version with each processing run.

---

## Oversight 9 — Current vs historical specifications

The specification requires current specifications but also requires version tracking.

Therefore:

```
current specification
```

and:

```
historical specification
```

should both be representable.

Never overwrite history.

---

## Oversight 10 — Human override

The agent may be wrong about:

```
deck
syllabus
mapping
card
```

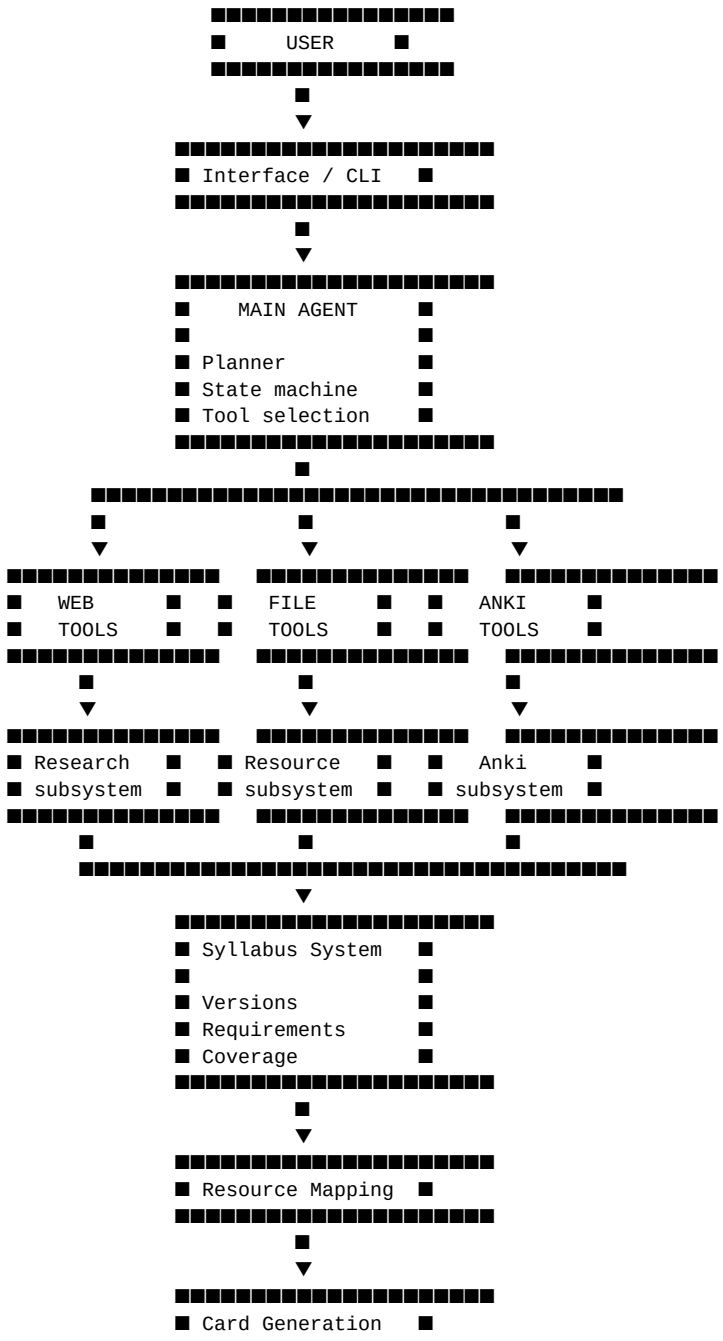
The user must be able to override these before writes.

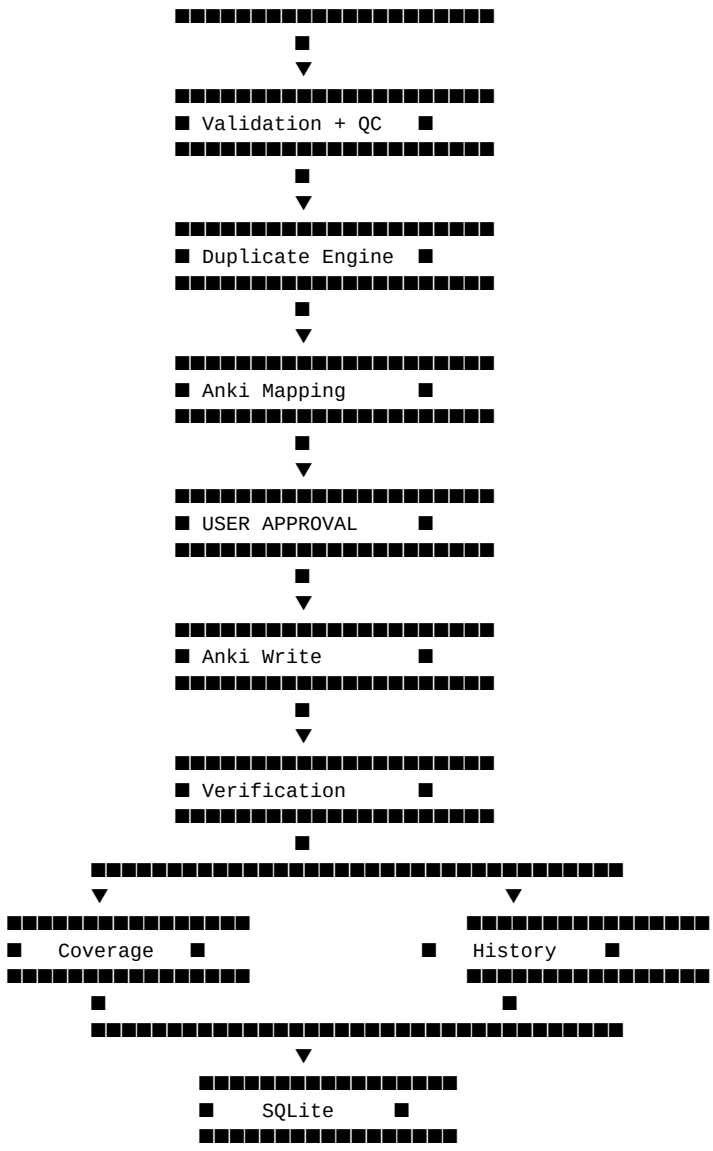
This is why the approval interface matters.

---

# 61. Final architecture diagram

Here is the architecture I would ultimately build:






---

# 62. The single most important dependency decision

If you remember only one thing from this roadmap, remember this:

```
    DON'T BUILD
        ↓
"the AI agent"
        ↓
    all at once
```

Instead:

```
Python
 ↓
Configuration
 ↓
Database
 ↓
Anki
 ↓
LLM
 ↓
```

```
Tools
 ↓
Files
 ↓
Research
 ↓
Syllabus
 ↓
Mapping
 ↓
Cards
 ↓
QC
 ↓
Duplicates
 ↓
Approval
 ↓
Agent
 ↓
Automation
```

The agent is actually **near the middle/end of the implementation**, not the first thing you should build.

The reason is that an agent without reliable tools is basically a chatbot with aspirations.

Your agent becomes useful because it has reliable capabilities underneath it.

---

# 63. Your first coding session — exactly what to do

This is where I recommend you start **right now**.

Do **not** build the agent yet.

Your first session should establish the project.

## Step 1 — Create the project

Create:

```
gcse-revision-agent
```

Inside it:

```
gcse-revision-agent/
■■■ src/
■■■ tests/
■■■ config/
■■■ data/
■■■ README.md
■■■ .gitignore
■■■ pyproject.toml
```

---

## Step 2 — Create your Python environment

From the VS Code terminal:

```
python -m venv .venv
```

Activate it.

On Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Then verify:

```
python --version
```

---

## Step 3 — Initialise Git

```
git init
```

Your `.gitignore` should include at least:

```
.venv/
.env
__pycache__/
*.pyc
data/cache/
```

---

## Step 4 — Create your first program

Create:

```
src/main.py
```

Initially, all it needs to do is prove Python can execute your application.

Conceptually:

```
Start application
 ↓
Print "GCSE Revision Agent"
 ↓
Exit
```

Run:

```
python src/main.py
```

Expected:

```
GCSE Revision Agent
```

---

### Step 5 — Make your first Git commit

```
git add .
git commit -m "Initial project setup"
```

This establishes your first known-good state.

---

## 64. Your first actual engineering milestone

After the basic setup, **do not jump to the LLM**.

Your first real target should be:

```
Python
   ↓
AnkiConnect
   ↓
getDeckNames
   ↓
Python prints your decks
```

Then:

```
Python
   ↓
AnkiConnect
   ↓
create test note
   ↓
verify test note
```

That is the first point where you're no longer just setting up a project—you are building the actual system.

The original project-development material specifically recommends this progression: establish Python → AnkiConnect → create a test card before introducing the AI. ■filecite■turn0file0■L45-L71■

---

## 65. Your first major milestone

I would define **Milestone 1** as:

> **The program can inspect Anki and safely create, inspect, modify and verify a test note.**

Then **Milestone 2**:

> **The program can ask an LLM for structured flashcards and validate the response.**

Then:

> **Milestone 3: LLM + Anki.**

Then:

> **Milestone 4: PDF + syllabus + cards.**

Then:

> **Milestone 5: research + mapping + QC + duplicates.**

Then:

> **Milestone 6: approval + history + coverage.**

Then:

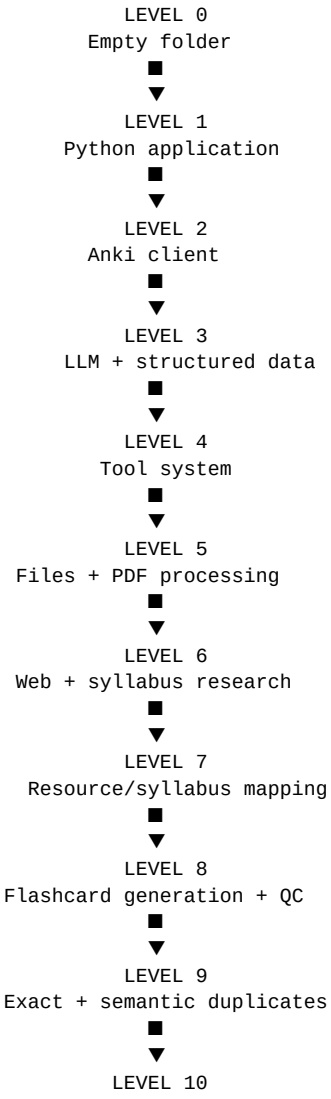
> **Milestone 7: complete autonomous pipeline.**

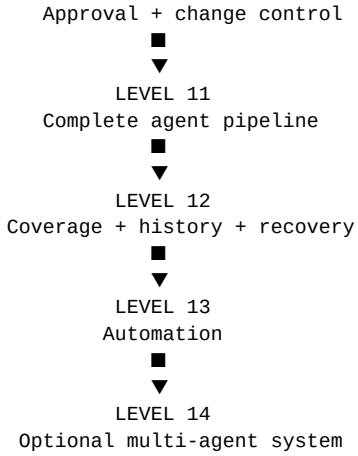
Then—and only then—consider multi-agent architecture.

---

## Bottom line

Your project should evolve like this:






**That is the roadmap I would use to actually build the project.**

And crucially, it preserves the specification's central goal: **official specification** → **relevant knowledge** → **legitimate resources/research** → **high-quality cards** → **duplicate/QC checks** → **correct Anki structure** → **user approval** → **verified Anki changes** → **coverage/history** → **eventual automation.**
■filecite■turn0file2■L542-L603■